



Pontifícia Universidade Católica do Paraná

Curso: **BSI**

Ano/Semestre: **2026/1**

Disciplina: **Experiência Criativa: Projetando Sistemas de Informação**

Professor: **Cleverson Avelino e Vinícius G. Mendonça**

Período: **5º**

Turma: **A / U**

Turno: **Noturno**

Aluno 1: Adrian Antônio de Souza Gomes

Aluno 2: Edmund Soares de Sousa

Aluno 3: Lucas Azzolin Haubmann

Aluno 4: Vinicius Lima Teider

Data: 25/04/2026

Desenvolvimento do Projeto



Curitiba — 2026

Sumário

1. Introdução.....	3
2. Concepção do Projeto.....	4
2.1 Descrição do Sistema.....	4
2.2 Justificativa da Problemática.....	4
2.3 Público-Alvo.....	5
2.4 Objetivos e Metas.....	5
3. Stack Tecnológica.....	6
3.1 Estrutura do Repositório.....	6
4. Requisitos Funcionais.....	7
5. Requisitos Não Funcionais.....	9
6. Cronograma de Desenvolvimento.....	10
Sprint 0 — 10/03 a 17/03/2026.....	10
Sprint 1 — 18/03 a 25/03/2026.....	10
Sprint 2 — 26/03 a 01/04/2026.....	11
Sprint 3 — 08/04 a 15/04/2026.....	11
Sprint 4 — 22/04 a 29/04/2026.....	12
Sprint 5 — 06/05 a 13/05/2026.....	12
Sprint 6 — 13/05 a 20/05/2026.....	13
Sprint 7 — 20/05 a 27/05/2026.....	13
Sprint 8 — 27/05 a 02/06/2026.....	14
7. Conclusão.....	15

1. Introdução

Este documento registra o desenvolvimento do Tukan Nutrition, plataforma web de nutrição personalizada com suporte de Inteligência Artificial, desenvolvida como projeto integrador da disciplina Experiência Criativa: Projetando Sistemas de Informação, do quinto período do Bacharelado em Sistemas de Informação da PUCPR, semestre 2026/1.

O projeto foi desenvolvido por uma equipe de quatro integrantes com início efetivo em 17 de março de 2026 e entrega em 2 de junho de 2026. O desenvolvimento seguiu práticas de Engenharia de Software: levantamento formal de requisitos, modelagem de banco de dados relacional, implementação integrada de frontend e backend com separação de responsabilidades, testes automatizados com cobertura monitorada, versionamento com Git e GitFlow, documentação de API com OpenAPI/Swagger e publicação em nuvem com pipelines de CI/CD via GitHub Actions.

A plataforma responde a uma necessidade real e documentada: a dificuldade de grande parte da população brasileira em transformar informações genéricas sobre alimentação em hábitos práticos, personalizados e compatíveis com sua rotina. O Tukan usa IA generativa como motor central de personalização, gerando planos alimentares semanais estruturados com base em dados individuais do usuário.

Este documento está organizado nas seguintes seções: concepção do projeto, requisitos funcionais e não funcionais, stack tecnológica, cronograma com registro detalhado das sprints e conclusão.

2. Concepção do Projeto

2.1 Descrição do Sistema

O Tukan é uma aplicação web full-stack de nutrição personalizada que transforma dados individuais em planos alimentares semanais práticos e adaptáveis. Utilizando IA generativa como motor de personalização — com as chaves do provedor mantidas exclusivamente no backend —, o sistema gera planos com múltiplas opções por refeição, detalha macronutrientes, acompanha a evolução do usuário ao longo do tempo e produz feedbacks textuais adaptativos baseados em dados reais de uso.

A solução é composta por três módulos principais:

- Módulo do usuário: jornada de onboarding (triagem nutricional), geração e visualização do plano alimentar, check-in diário de refeições, registro de peso e acesso a relatórios de progresso com gráficos e avaliações por IA.
- Módulo de IA: integração backend-only com provedor de IA configurável via variáveis de ambiente (AI_PROVIDER, AI_MODEL); validação e sanitização do retorno JSON antes do processamento.
- Módulo administrativo: painel de gestão de usuários (listar, editar, bloquear, banir), controle de limite de gerações por ciclo semanal, monitoramento de uso e reset de credenciais com envio automático por e-mail.

A arquitetura segue o modelo SPA estática + API stateless: o frontend Vue 3 é servido como bundle estático via CDN, enquanto o backend Spring Boot expõe uma API RESTful com configuração externalizada por ambiente. A comunicação entre as camadas ocorre exclusivamente via HTTPS/JSON de forma assíncrona.

O nome Tukan referencia o tucano, ave que transmite leveza, inteligência e adaptação, refletido no logotipo geométrico e minimalista que combina modernidade e acessibilidade.

2.2 Justificativa da Problemática

A justificativa do Tukan apoia-se em dados concretos sobre os hábitos alimentares da população brasileira. Segundo a Pesquisa Nacional de Saúde (PNS, 2019), entre 63% e 68% dos moradores das capitais não consomem frutas e hortaliças regularmente. Pesquisa do Opinion Box (2023) aponta que 29% dos brasileiros citam a falta de tempo para cozinhar como principal obstáculo a uma alimentação saudável. O custo médio de uma consulta com nutricionista, entre R\$ 150 e R\$ 350 por sessão sem cobertura de plano de saúde, cria uma barreira real de acesso para grande parte da população.

Embora o volume de conteúdo sobre nutrição disponível na internet seja imenso, a maior parte das informações é genérica, contraditória ou descontextualizada. Aplicativos existentes como MyFitnessPal e Lifesum oferecem baixa personalização e não consideram fatores como orçamento semanal, rotina irregular ou condições clínicas específicas do usuário.

O Tukan se diferencia ao integrar dados biométricos, clínicos, financeiros e comportamentais para entregar recomendações genuinamente personalizadas e adaptativas; posicionando-se como alternativa digital acessível ao acompanhamento nutricional presencial.

2.3 Público-Alvo

O público principal é composto por homens e mulheres entre 18 e 55 anos, residentes em centros urbanos brasileiros, das classes B e C (renda entre R\$ 1.200 e R\$ 9.000/mês), com acesso a smartphone e internet. Esse segmento inclui estudantes universitários, trabalhadores com rotina intensa, mães em pós-parto, pessoas com condições clínicas como diabetes tipo 2 ou hipertensão e praticantes de atividade física que buscam orientação nutricional prática sem acesso regular a nutricionistas.

O público secundário inclui nutricionistas que buscam ferramentas digitais confiáveis para complementar o acompanhamento entre consultas, e adolescentes atletas (15–17 anos) com mediação dos responsáveis.

Segundo pesquisa da UERJ (Mello et al., 2019), o gênero predominante entre usuários de aplicativos de nutrição é o feminino (61%), com faixa etária mais ativa entre 20 e 40 anos.

2.4 Objetivos e Metas

Objetivo geral: desenvolver e entregar um MVP funcional de plataforma web de nutrição personalizada com IA, seguindo as boas práticas de Engenharia de Software, até o fim do semestre 2026/1.

Objetivos específicos:

- Implementar autenticação JWT com dois perfis distintos (USER e ADMIN), com validação no frontend e no backend;
- Construir triagem nutricional em múltiplas etapas com validação completa antes da geração do plano;
- Integrar provedor de IA generativa no backend para geração de planos alimentares semanais e avaliações de progresso;
- Desenvolver dashboard analítico com gráficos de consumo calórico, macronutrientes, evolução de IMC e aderência semanal;
- Garantir separação total entre frontend (Vue 3 SPA) e backend (Spring Boot REST API), com comunicação assíncrona via JSON;
- Atingir cobertura de testes automatizados superior a 75% em frontend (Vitest) e backend (JUnit / Spring Boot Test);
- Publicar o sistema em ambiente de nuvem com pipelines de CI/CD (GitHub Actions) configurados por ambiente (dev, teste, prod);
- Documentar 100% dos endpoints com OpenAPI/Swagger, com descrições, erros e exemplos reais de payload.

3. Stack Tecnológica

A stack foi definida para garantir separação de responsabilidades, rastreabilidade e alinhamento com as boas práticas de Engenharia de Software exigidas na disciplina. O repositório é estruturado como mono-repo público no GitHub, com os diretórios frontend/ e backend/ independentes e pipelines separados para cada camada.

Camada	Tecnologia	Função no sistema
Frontend	Vue 3 + TypeScript	Interface SPA com tipagem estática; componentes reutilizáveis e contratos de API definidos em tipos
Frontend	Vite	Dev server e pipeline de build de produção; transpilação e tree-shaking
Backend	Spring Boot (Java 17+)	API RESTful; regras de negócio; orquestração das chamadas ao provedor de IA
Backend	Maven / Gradle Wrapper	Build, testes, empacotamento JAR e geração de imagem OCI via Cloud Native Buildpacks
Banco de Dados	PostgreSQL	Persistência relacional; índices de performance; stored procedures para relatórios
Autenticação	JWT	Tokens stateless; dois perfis de permissão (USER / ADMIN); validação no frontend e no backend
IA	API externa (configurável)	Geração e análise de planos alimentares; chave mantida exclusivamente no backend
CI/CD	GitHub Actions	Pipelines por ambiente (dev / teste / prod); quality gates; deploy automatizado
Testes Frontend	Vitest	Testes unitários e de componente; cobertura $\geq 75\%$
Testes Backend	Spring Boot Test / JUnit	Testes de unidade e integração; cobertura $\geq 75\%$
Documentação API	OpenAPI / Swagger	Documentação online com payloads reais, modelos de dados e erros documentados
Hospedagem	Nuvem (estático JVM/OCI) +	Frontend como SPA estática via CDN; backend como JAR ou imagem OCI

3.1 Estrutura do Repositório

O mono-repo segue a estrutura padrão para projetos Vue 3 + Spring Boot:

- frontend/src/ — components/, composables/, router/, services/, stores/, types/, views/
- backend/src/main/java/ — controllers, services, repositories, models, config
- backend/src/main/resources/application.yml — configuração externalizada por perfil
- .github/workflows/ — pipelines de CI/CD por ambiente
- docs/ — documentação adicional e diagramas

4. Requisitos Funcionais

Os 20 requisitos funcionais foram levantados na fase de concepção e utilizados como base para o planejamento das sprints, a construção do backlog no Trello e a execução da implementação. A coluna de prioridade reflete a ordem de entrega no MVP: RF001–RF019 compõem o escopo principal; RF020 é requisito extra (MVP+).

ID	Requisito Funcional	Descrição
RF001	Definição de Limites de Uso	O administrador pode definir o número máximo de gerações de planos via IA por usuário dentro do ciclo semanal, controlando custos e estabilidade do serviço.
RF002	Controle de Acessos	O administrador pode listar, editar, bloquear (temporariamente) ou banir (permanentemente) usuários da plataforma.
RF003	Gestão de Credenciais	O administrador pode realizar o reset de senhas de usuários; o sistema envia automaticamente a nova credencial por e-mail via serviço de SMTP.
RF004	Monitoramento de Uso	O sistema fornece ao administrador um painel com volume total de dietas geradas e quantidade de usuários ativos por período.
RF005	Autenticação de Usuário	O sistema permite cadastro e login de usuários com e-mail e senha; sessão gerenciada via token JWT com dois perfis de permissão distintos (usuário e administrador).
RF006	Refinamento Biométrico	O sistema coleta dados obrigatórios de idade (data de nascimento), altura, peso e sexo para cálculos metabólicos.
RF007	Definição de Metas	O usuário escolhe entre emagrecimento ou ganho de massa, definindo peso-alvo e tempo de dedicação ao plano.
RF008	Perfil de Rotina	O sistema registra a quantidade de refeições diárias (entre 3 e 5), a prática de jejum e o orçamento semanal máximo disponível para alimentação.
RF009	Restrições de Saúde	O usuário pode informar condições de saúde (ex.: diabetes), alergias e preferências alimentares (onívora ou vegetariana).
RF010	Trava de Segurança	O sistema valida o preenchimento completo de todos os campos da triagem antes de habilitar o botão de geração do plano alimentar.
RF011	Geração via IA	O sistema envia os dados da triagem ao provedor de IA configurado no backend e persiste o retorno estruturado em formato JSON; exporta CSV para o banco de dados.
RF012	Opções por Refeição	O plano semanal apresenta entre 2 e 3 sugestões distintas para cada refeição (café da manhã, almoço, lanche, jantar e ceia).
RF013	Detalhamento de Macros	Cada sugestão de refeição exibe título, porção sugerida, calorias totais e gramatura de proteínas e carboidratos.
RF014	Registro de Peso	O usuário seleciona uma data no calendário e registra seu peso atual para fins de acompanhamento histórico.
RF015	Submit Diário	O usuário realiza check-in diário das refeições consumidas, limitado às opções previamente geradas no plano semanal ativo.

RF016	Relatório de Consumo	O sistema gera gráficos de consumo de calorias e macronutrientes com base em filtros de período selecionados pelo usuário.
RF017	Evolução de IMC	O sistema calcula e exibe graficamente a evolução do Índice de Massa Corporal ao longo do tempo, com base nos pesos registrados.
RF018	Avaliação de Progresso via IA	O sistema utiliza o provedor de IA para analisar o peso inicial versus o atual e gerar um texto avaliativo contextualizado sobre a progressão das metas.
RF019	Persistência de Avaliações	Cada texto avaliativo gerado pela IA é armazenado no histórico do usuário para consultas futuras.
RF020	Favoritos e Substituições	O usuário pode favoritar opções de refeições; a IA prioriza os favoritos na geração dos ciclos semanais subsequentes. (Requisito extra — MVP+)

5. Requisitos Não Funcionais

Os requisitos não funcionais definem os atributos de qualidade, desempenho, segurança e arquitetura do sistema. Eles orientaram as decisões técnicas de stack, infraestrutura e padrões de codificação ao longo de todo o desenvolvimento.

ID	Categoria	Descrição
RNF001	Frontend	Interface desenvolvida em Vue 3 + TypeScript com Vite; responsiva em desktop e mobile, com suporte nativo a dark mode e internacionalização (i18n).
RNF002	Backend	API RESTful implementada em Spring Boot (Java 17+); verbos e códigos HTTP semânticos, mensagens de erro padronizadas, configuração externalizada via variáveis de ambiente.
RNF003	Persistência	Dados armazenados em PostgreSQL com índices nas colunas de filtragem frequente (data, usuário) e stored procedures para consultas complexas de relatório.
RNF004	Performance	Tempo de resposta da geração de plano via IA não superior a 15 segundos em condições normais; endpoints REST respondem em menos de 500 ms para operações de CRUD.
RNF005	Segurança	Comunicação via HTTPS; senhas armazenadas com BCrypt; autenticação via JWT com pelo menos dois perfis (USER e ADMIN); chaves de IA mantidas exclusivamente no backend.
RNF006	CI/CD	Pipelines GitHub Actions por ambiente (dev, teste, prod); etapas: lint → type-check → testes → build → publicação; deploy bloqueado por falha em qualquer gate de qualidade.
RNF007	Acessibilidade	Interface aderente às diretrizes WCAG 2.1 nível AA para suporte a diferentes perfis de usuários.
RNF008	Escalabilidade	Arquitetura stateless no backend suporta até 500 acessos simultâneos sem degradação perceptível; frontend servido como SPA estática via CDN.
RNF009	Tratamento de Dados IA	JSON retornado pelo provedor de IA validado e sanitizado no backend antes do processamento, evitando erros na importação de macronutrientes.
RNF010	Arquitetura	Mono-repo com separação clara de frontend/ e backend/; backend organizado em camadas (Controller → Service → Repository); modular para expansões futuras.
RNF011	UX / Feedback Visual	Componentes de loader e skeleton durante o processamento da IA; mensagens informativas em caso de erro; transições suaves entre estados da interface.
RNF012	Versionamento	Repositório GitHub com GitFlow; branches por desenvolvedor; merge antes de cada sprint; versionamento semântico a cada ciclo; commits descritivos e rastreáveis.

6. Cronograma de Desenvolvimento

O desenvolvimento foi organizado em sprints semanais a partir de 17 de março de 2026, com entrega final em 2 de junho de 2026. As atividades foram gerenciadas no quadro kanban do Trello, com backlog estruturado pelas tarefas definidas na Sprint 0. O versionamento seguiu GitFlow com branches individuais por desenvolvedor, merge antes de cada sprint, versionamento semântico (v1.0.0, v1.1.0...) e commits descritivos a cada entrega de funcionalidade.

As sprints 1 a 7 cobrem o desenvolvimento das funcionalidades do sistema. A Sprint 8 é dedicada exclusivamente a testes integrados, correções, revisão de cobertura e preparação final para entrega.

Sprint 0 — 10/03 a 17/03/2026

Objetivo:

Apresentação da disciplina, definição do projeto e estruturação do ambiente de trabalho.

Integrante	Atividades realizadas
Todos os integrantes	Apresentação e entendimento dos critérios de avaliação da disciplina; alinhamento sobre o escopo e as prioridades do projeto Tukan; definição do tema e do MVP; construção do quadro kanban no Trello com todas as tarefas do semestre; levantamento, discussão e validação dos 20 requisitos funcionais e 12 não funcionais; definição do cronograma de sprints e da divisão de responsabilidades entre os integrantes; criação do repositório GitHub com estrutura GitFlow.

Dificuldades: Definir o escopo do MVP dentro das restrições de prazo do semestre, especialmente em relação à profundidade da integração com IA.

Soluções: Realização de reunião de alinhamento para priorizar os requisitos do núcleo funcional (RF005–RF019) e marcar RF020 como extra (MVP+), garantindo entregabilidade dentro do prazo.

Sprint 1 — 18/03 a 25/03/2026

Objetivo:

Configuração do ambiente, definição da identidade visual e modelagem do banco de dados.

Integrante	Atividades realizadas
Adrian Gomes	Pesquisa e avaliação de templates de frontend Vue 3 com responsividade mobile (Android e iOS); seleção e levantamento de ao menos 5 opções para deliberação em equipe; configuração inicial do repositório GitHub com estrutura GitFlow e branches individuais; configuração do arquivo README.md.
Edmund Sousa	Modelagem do banco de dados PostgreSQL: levantamento de entidades, atributos e relações; elaboração do diagrama ER; definição de chaves primárias, estrangeiras e índices nas colunas de filtragem frequente (user_id, date).
Lucas Haubmann	Configuração do ambiente de backend Spring Boot: estrutura de pacotes (controller / service / repository), dependências Maven/Gradle, conexão com PostgreSQL e perfis de configuração por ambiente (dev / prod); primeiro commit de estrutura do projeto.
Vinicius Teider	Definição e documentação da identidade visual do Tukan (paleta de cores, tipografia, componentes base); configuração inicial do projeto Vue 3 + TypeScript + Vite; estrutura de diretórios do frontend (components/, composables/, router/, services/, stores/, types/, views/).

Dificuldades: Dificuldade em alinhar a identidade visual com os templates disponíveis e com as capacidades reais do Vue 3, especialmente em relação ao suporte nativo a dark mode.

Soluções: A equipe deliberou em conjunto sobre o template mais adequado e definiu um sistema de design simples e escalável, com suporte a dark mode implementado via variáveis CSS globais desde o início.

Sprint 2 — 26/03 a 01/04/2026

Objetivo:

Controle de acesso, autenticação JWT e estrutura base do frontend.

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento das telas de login e cadastro no frontend (RF005): formulários com validação, regex e feedback visual de erro; integração com o contexto de autenticação; roteamento protegido por perfil.
Edmund Sousa	Criação das tabelas do banco de dados PostgreSQL com as migrações iniciais; definição das constraints e índices; validação do modelo relacional com dados de teste.
Lucas Haubmann	Implementação do módulo de autenticação no backend: endpoint de login e cadastro, geração e validação de tokens JWT com dois perfis (USER / ADMIN), retorno dos códigos HTTP semânticos corretos; configuração de CORS para o frontend.
Vinicius Teider	Estruturação do layout principal do frontend: menu lateral responsivo, sistema de rotas com Vue Router, contexto global de autenticação com Pinia; primeiros componentes reutilizáveis; configuração do dark mode via variáveis CSS.

Dificuldades: Configurar a validação JWT de forma consistente nos dois lados da aplicação — frontend e backend — e retornar os status HTTP corretos em todos os cenários de erro exigiu cuidado especial.

Soluções: Foram definidos padrões de resposta de API no início da sprint (envelope de erro padronizado com status code, message e timestamp), documentados imediatamente no OpenAPI para guiar o desenvolvimento integrado.

Sprint 3 — 08/04 a 15/04/2026

Objetivo:

Onboarding e triagem nutricional completa (completar perfil).

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento do formulário de triagem no frontend (RF006 a RF010): quatro etapas sequenciais (biometria, metas, rotina, restrições e preferências); trava que bloqueia a geração do plano enquanto campos obrigatórios estiverem incompletos; feedback visual de progresso entre etapas; validações client-side.
Edmund Sousa	Implementação dos endpoints de backend para recebimento e persistência dos dados de triagem; validações de regras de negócio (ex.: refeições diárias entre 3 e 5, campos obrigatórios); retorno de mensagens de erro claras com status HTTP adequados.
Lucas Haubmann	Modelagem da integração com o provedor de IA: definição do formato do payload de entrada (dados da triagem em JSON) e do retorno esperado (plano semanal estruturado); primeiros testes de comunicação com a API do provedor; configuração das variáveis de ambiente AI_PROVIDER e AI_MODEL.
Vinicius Teider	Componentização das etapas do onboarding no frontend; implementação das transições suaves entre etapas; revisão de responsividade em mobile; testes de usabilidade manual do fluxo completo de triagem.

Dificuldades: A modelagem do payload enviado ao provedor de IA e o tratamento do retorno JSON demandaram múltiplas iterações para garantir que os dados chegassem no formato correto e fossem sanitizados antes do processamento (RNF009).

Soluções: Foi criado um módulo dedicado de validação e sanitização do JSON no backend, com tratamento de exceções específicas para retornos malformados e mensagens de erro claras para o usuário final.

Sprint 4 — 22/04 a 29/04/2026

Objetivo:

Geração de plano alimentar via IA e visualização completa do plano no frontend.

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento da tela de plano alimentar no frontend (RF011 a RF013): exibição das opções geradas pela IA com macronutrientes detalhados; botão e lógica de troca de opção de refeição; painel lateral com justificativa do plano gerado pela IA ('por que este plano combina com você').
Edmund Sousa	Implementação do endpoint de geração de plano no backend: recebimento dos dados da triagem, envio ao provedor de IA, validação e persistência do retorno; lógica de controle do limite semanal de gerações por usuário (RF001) com retorno de status 429 ao atingir o limite.
Lucas Haubmann	Refinamento do módulo de integração com a IA: ajuste de prompt, tratamento de falhas de timeout e retornos inválidos com fallback; criação do banco de alimentos pré-configurados para otimização do consumo de tokens; primeiros testes automatizados do serviço de IA.
Vinicius Teider	Implementação de skeleton loader e loader spinner durante o processamento da IA (RNF011); testes de ponta a ponta do fluxo triagem → geração → visualização do plano; ajustes de layout com base nos resultados dos testes; atualização da documentação OpenAPI com os novos endpoints.

Dificuldades: O tempo de processamento do provedor de IA variou significativamente entre as requisições, superando o limite de 15 segundos em alguns cenários, especialmente com payloads maiores.

Soluções: Foram aplicadas otimizações no prompt e implementado um banco de alimentos pré-configurados para reduzir o volume de tokens por requisição. O limite de 15 segundos passou a ser atendido na grande maioria dos casos, com feedback visual adequado nos casos de processamento mais longo.

Sprint 5 — 06/05 a 13/05/2026

Objetivo:

Registro diário de refeições e homepage do usuário.

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento da homepage do usuário: menu lateral funcional, mini dashboard com frequência de registros e aderência semanal, atalhos rápidos para as principais funcionalidades, inserção dos textos avaliativos gerados pela IA.
Edmund Sousa	Implementação dos endpoints de registro diário no backend (RF015): persistência do check-in de refeições restrita às opções do plano ativo; CRUD completo com verbos HTTP corretos (GET, POST, PUT, DELETE); retorno de status 422 para tentativas de registro fora do plano.
Lucas Haubmann	Desenvolvimento da tela de registro diário no frontend: componente de calendário para seleção de data, exibição das refeições disponíveis no dia, saldo calórico diário dinâmico; tela de leitura para dias já submetidos com opção de edição.

Vinicius Teider	Construção do dashboard de aderência semanal com gráfico de barras e filtro de período; integração dos dados de consumo calórico diário; revisão de responsividade e dark mode nos componentes desenvolvidos; início da configuração dos testes unitários com Vitest.
------------------------	---

Dificuldades: A lógica de estado do calendário de registro — especialmente a distinção entre dias futuros, dias com registro e dias sem registro — gerou complexidade no gerenciamento de estado do frontend.

Soluções: O estado de data e plano ativo foi centralizado em uma store Pinia dedicada, simplificando o acesso e a reatividade entre componentes e eliminando inconsistências de estado nos casos de navegação entre datas.

Sprint 6 — 13/05 a 20/05/2026

Objetivo:

Módulo de progresso: registro de peso, evolução de IMC, relatórios e feedback adaptativo via IA.

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento das telas de progresso no frontend (RF016 a RF019): gráficos de evolução de IMC, relatórios de consumo calórico e de macronutrientes por período com filtros de data; tela de histórico de avaliações geradas pela IA.
Edmund Sousa	Implementação dos endpoints de progresso no backend: cálculo dinâmico do IMC com base nos pesos registrados e na altura da triagem; agregação de dados de consumo por período com stored procedures PostgreSQL; persistência das avaliações da IA no histórico do usuário.
Lucas Haubmann	Integração do módulo de avaliação de progresso com a IA (RF018): modelagem do prompt comparativo (peso inicial × atual × histórico de aderência), tratamento do retorno textual e persistência; otimização das queries de relatório com índices e stored procedures para garantir performance (RNF003).
Vinicius Teider	Construção da tela de registro de peso com calendário e histórico de pesagens (RF014); testes de integração do fluxo de progresso completo; ajustes de UX no layout dos gráficos com base nos testes; aumento da cobertura de testes Vitest nos componentes de progresso.

Dificuldades: A geração de relatórios com filtros de data sobre grandes volumes de registros apresentou degradação de performance nas primeiras implementações.

Soluções: Foram implementadas stored procedures no PostgreSQL para as três consultas de relatório mais pesadas, com adição de índices compostos nas colunas user_id + date, garantindo tempo de resposta adequado conforme RNF003 e RNF004.

Sprint 7 — 20/05 a 27/05/2026

Objetivo:

Homepage administrativa, CI/CD, documentação de API e internacionalização.

Integrante	Atividades realizadas
Adrian Gomes	Desenvolvimento da homepage administrativa no frontend (RF001 a RF004): listagem paginada de usuários com busca e filtros; ações de bloquear, banir e editar usuário; painel de monitoramento com volume de dietas geradas e usuários ativos por período.
Edmund Sousa	Implementação dos endpoints administrativos no backend: CRUD de usuários com controle de status (ATIVO / BLOQUEADO / BANIDO); monitoramento de uso; reset de senha com geração automática de credencial e envio via SMTP (RF003); retorno de status 403 para usuários bloqueados ou banidos.

Lucas Haubmann	Configuração dos pipelines de CI/CD com GitHub Actions: jobs separados para frontend e backend; etapas lint → type-check → testes → build → deploy por ambiente (dev, teste, prod); configuração de secrets para credenciais de banco e chave de IA; health checks e monitoramento de disponibilidade.
Vinicius Teider	Implementação da internacionalização do frontend (i18n): textos estáticos e conteúdo de páginas em PT-BR e EN; formatação de datas, números e valores monetários conforme localização; ajustes finais de responsividade em dispositivos móveis e dark mode; finalização da documentação OpenAPI com exemplos reais de payload para todos os endpoints.

Dificuldades: A configuração dos pipelines de CI/CD com múltiplos ambientes e segredos distintos exigiu atenção para evitar que credenciais de produção fossem expostas em jobs de desenvolvimento.

Soluções: Foram criados arquivos de secrets separados por ambiente no GitHub Actions e adotada a convenção de nenhuma credencial sensível em variáveis VITE_*, mantendo chaves de banco e de IA exclusivamente no backend conforme RNF005.

Sprint 8 — 27/05 a 02/06/2026

Objetivo:

Testes integrados, cobertura, ajustes finais e preparação para entrega.

Integrante	Atividades realizadas
Adrian Gomes	Execução dos testes de integração e end-to-end no frontend; verificação e complementação da cobertura de testes Vitest (meta ≥ 75%); ajustes de interface e responsividade identificados nos testes; validação do dark mode e da internacionalização em dispositivos móveis reais.
Edmund Sousa	Execução dos testes de unidade e integração no backend com JUnit; verificação da cobertura (meta ≥ 75%); correção de bugs identificados; revisão final dos status HTTP em todas as rotas; atualização e publicação da documentação OpenAPI com os ajustes finais.
Lucas Haubmann	Testes de carga para verificação de escalabilidade (até 500 acessos simultâneos); validação dos pipelines de CI/CD nos três ambientes com deploy real; revisão do histórico de commits e do versionamento semântico no repositório; garantia de que todos os integrantes têm contribuições registradas no histórico do GitHub.
Vinicius Teider	Revisão e finalização da documentação do projeto; organização dos arquivos README e docs/ no repositório; produção dos diagramas UML finais (diagrama de classes, dois diagramas de sequência e diagrama de atividades); consolidação de todos os artefatos para entrega.

Dificuldades: A concentração de grande parte dos commits no repositório em um único integrante ao longo do semestre gerou desequilíbrio no histórico de contribuições do GitHub.

Soluções: Na sprint final, os commits pendentes de funcionalidades desenvolvidas localmente foram organizados e registrados por cada integrante responsável, garantindo rastreabilidade fiel das contribuições reais de cada membro da equipe.

7. Conclusão

O desenvolvimento do Tukan Nutrition ao longo do semestre 2026/1 representou uma experiência completa de Engenharia de Software aplicada a um problema real. Partindo de uma proposta conceitual consolidada em semestres anteriores — com artefatos de Design Thinking, Lean Inception, modelagem BPMN, user stories e casos de uso —, a equipe evoluiu do planejamento para um sistema funcional com frontend e backend integrados, autenticação segura, IA generativa, dashboard analítico, testes automatizados e publicação em nuvem com pipelines de CI/CD.

Do ponto de vista técnico, o projeto atendeu aos critérios fundamentais da disciplina: separação total entre frontend Vue 3 SPA e backend Spring Boot com comunicação assíncrona via JSON; CRUD completo com persistência em PostgreSQL e verbos HTTP semânticos; autenticação JWT com dois perfis distintos; dashboard com gráficos e filtros analíticos; cobertura de testes $\geq 75\%$ em ambas as camadas; documentação de API com OpenAPI/Swagger online; pipelines GitHub Actions por ambiente; responsividade com dark mode; e internacionalização da interface.

O uso de IA generativa como tecnologia adicional ao escopo base da disciplina, com orquestração exclusivamente no backend, chaves de provedor externalizadas via variáveis de ambiente e sanitização rigorosa dos retornos JSON, reforça o posicionamento do Tukan como solução moderna, segura e alinhada às boas práticas de sistemas que consomem serviços de IA em produção.

Do ponto de vista do produto, o Tukan consolidou sua proposta de valor, 'Seu plano alimentar se adapta à sua vida. Não o contrário.', ao entregar uma plataforma que vai além do gerador de dietas, oferecendo acompanhamento contínuo, adaptativo e baseado em dados reais. Como próximos passos naturais, a equipe identifica: validação com usuários reais, refinamento iterativo do módulo de IA, análise de viabilidade de lançamento com modelo de negócio freemium e parceria com profissionais de nutrição para validação técnica das recomendações geradas.

— Curitiba, junho de 2026 —